Drop, Reflect and Reuse a Quantum Temporary

ANONYMOUS AUTHOR(S)

Compilers often uncompute a temporary before reusing it, running its gates backwards. We ask instead what the temporary ends as. A drop is safe when, just before it, no observer can tell the program from one that prepared the temporary without the input. This holds exactly when the temporary ends in one fixed pure state for every input, and then one gate on it alone restores its starting state. The test extends to each sequence of measurement outcomes with its weight and to borrowed registers. A prototype compiler computes the gate and reuses the freed wire.

1 Specifying a safe drop

Quantum data has by now a mature treatment in programming languages, from categorical semantics to typed and circuit languages [1, 19, 21, 22]. Recent lines of work build a pure, reversible core and add measurement to it as a computational effect [5, 10, 23]. This work is about the allocation and deallocation of registers holding quantum data. Consider two programs on a qubit v:

```
prog a (v : [2]) : [2] {          prog b (v : [2]) : [2] {
  new t : [2] @ |0>;                 new t : [2] @ |0>;
  H t;                               H t; CX t v;
  drop t; return v }                 drop t; return v }
```

In program (a), H puts t in $|+\rangle = (|0\rangle + |1\rangle)/\sqrt{2}$ whatever v held, and another H restores $|0\rangle$ without touching v. Program (b) adds `CX t v`, flipping v when t is 1, so at input $|0\rangle_v$ it produces the Bell state $(|00\rangle + |11\rangle)/\sqrt{2}$, which independent operations cannot prepare, so dropping t leaves v mixed. Both lexical lifetimes end at the drop, so checking scopes alone accepts both. Compilers usually uncompute a temporary [2, p. 525][3, 4, 17]. Silq and Unqomp automate this by letting only `qfree` gates, which create no superposition, produce the temporary [4, §4.4][17, §2.2], so both refuse program (a), whose one H is not `qfree` [17, §6.2]. We ask when one unitary on the dropped registers alone restores their allocation state and leaves the rest intact.

Now take an arbitrary program p that allocates a temporary t in a pure state ψ, works on it and the input v together with measurement-free code $M_{t\otimes v}$, and drops it. Beside p we put an independent witness $\mathcal{S}(R,U)$ that prepares t without consulting the input, by one gate R_t on t alone, and computes on v separately, by a gate U_v on v and fresh registers for any further temporaries of $M_{t\otimes v}$:

$$p(v) = \texttt{new t : [d] @ } \psi\texttt{; } M_{t\otimes v}\texttt{; drop t; return v},$$

$$\mathcal{S}(R,U)(v) = \texttt{new t : [d] @ } \psi\texttt{; } R_t\texttt{; } U_v\texttt{; drop t; return v}.$$

We call the drop safe if $p \equiv \mathcal{S}(R,U)$ for some fixed R and U, the same pair for every input x of v, where $p \equiv q$ says that no closed context tells p from q just before the drop, every register still visible. Throughout, registers are finite-dimensional over $\mathbb{C}$, and programs and contexts may apply any unitary.

2 Checking the equivalence

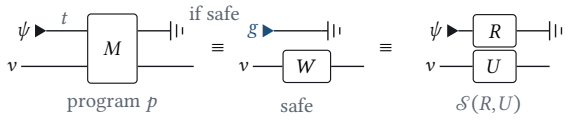
Stop the program just before the `drop` at hand, which we call its *site*. Delete earlier drops, which changes no verdict, and rename reused names apart. This *exposed prefix* is one map K, taking input x to the joint state with the dropped registers, the *block*, first.
Program (a) gives $K_{(a)}x = |+\rangle_t \otimes x_v = (|0\rangle_t \otimes x_v + |1\rangle_t \otimes x_v)/\sqrt{2}$,
and program (b) gives $K_{(b)}x = (|0\rangle_t \otimes x_v + |1\rangle_t \otimes X_v x_v)/\sqrt{2}$.

$$K_{(a)} = \overbrace{\frac{1}{\sqrt{2}}\begin{bmatrix}1&0&1&0\\0&1&0&1\\1&0&-1&0\\0&1&0&-1\end{bmatrix}}^{H\otimes I} \cdot \overbrace{\begin{bmatrix}1&0\\0&1\\0&0\\0&0\end{bmatrix}}^{|0\rangle\otimes I} = \overbrace{\frac{1}{\sqrt{2}}\begin{bmatrix}1&0\\0&1\\1&0\\0&1\end{bmatrix}}^{K_{(a)}} \qquad \overbrace{\frac{1}{\sqrt{2}}\begin{bmatrix}1&0\\0&1\\0&1\\1&0\end{bmatrix}}^{K_{(b)} = \mathrm{CX}\,K_{(a)}}$$

$K_0 = K_1 = I/\sqrt{2}$ $\qquad$ $K_0 = I/\sqrt{2},\quad K_1 = X/\sqrt{2}$

The matrix $|0\rangle \otimes I$ acts first, on the input x. Reading t in its basis $|0\rangle, |1\rangle$, the part of K tagged by $|i\rangle$ is a matrix of its own, the *slice* K_i, blue for $i = 0$ and gold for $i = 1$, which is what the program has done to v in the branch where t reads i, so that $Kx = \sum_i |i\rangle_t \otimes K_i x_v$.

The drop applies no gate and removes only the name t, so the hidden register remains, and we are asking whether it is left in one fixed pure state, independent of the input and uncorrelated with the rest. A context supplies the input, possibly entangled with a reference register of its own, may apply gates, and measures every register jointly with that reference, so contexts detect any correlation of t with other registers. The action of $M_{t\otimes v}$ on $\psi \otimes x$ factors at a safe drop, whatever its code, because every branch does the same thing to the remaining registers, here v, up to a possibly zero scalar, so the slices are proportional, $K_i = g_i W$ (Kraus rank at most one, a standard condition), where the g_i are the coordinates of one fixed pure state g of the temporary and W is what the prefix did to the rest. In program (a) both slices are $I_v/\sqrt{2}$, so $g = |+\rangle_t$ and $W = I_v$, while in program (b) they are $I_v/\sqrt{2}$ and $X_v/\sqrt{2}$, which are not proportional. A safe program, its factorization and its witness agree until the drop, as pictured: triangles prepare, ground symbols drop.

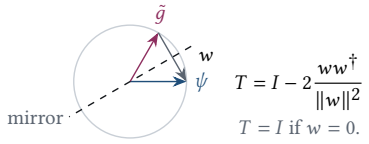


Theorem 2.1 (without measurement). *For a well-typed measurement-free program p at a drop of only allocated temporaries, let K be its exposed prefix's joint map and ψ the pure allocation state. The following are equivalent.*

(1) $p \equiv \mathcal{S}(R,U)$ for some independent witness.
(2) There are a unit vector g on the block and a linear map W to all other registers such that $\forall x,\ Kx = g \otimes Wx$.
(3) One unitary on the block alone restores ψ at every input.

The map W is an isometry, preserving norms and inner products. The witness prepares $g = R_t\psi$ and realizes W through U_v acting on the input and fresh registers.

Deutsch–Jozsa shows why slices need only be proportional, not equal. With $|-\rangle = (|0\rangle - |1\rangle)/\sqrt{2}$, its oracle obeys $O_f(|-\rangle \otimes |z\rangle) = |-\rangle \otimes (-1)^{f(z)}|z\rangle$, so the temporary stays fixed and the data carries the function's phase, and t's two slices differ only in sign.



We call g a *certificate*. Since $Kx = g \otimes Wx$ fixes g only up to a phase, choose the representative $\tilde{g}$ with $\langle\psi, \tilde{g}\rangle$ real and nonnegative, moving the phase into W. If the overlap is zero, make the first nonzero coordinate of $\tilde{g}$ real and positive. If $\tilde{g} = \psi$, no gate is needed, which is what safe use [14, Eq. (1)] and clean safety [13, Def. 1] require of every clean ancilla. Otherwise the pictured Householder reflection [12, p. 340, Eqs. (3)–(4)] $T = I - 2ww^\dagger/\|w\|^2$ through the hyperplane orthogonal to $w = \psi - \tilde{g}$ exchanges $\tilde{g}$ and ψ. Applied to the temporary alone, T returns it to ψ at every input and leaves the rest unchanged: $\|w\|^2 = -2\langle w, \tilde{g}\rangle$ gives $T\tilde{g} = \tilde{g} + w = \psi$. Being its own inverse, it is also a witness's R_t, as $T\psi = \tilde{g}$. Conversely, in any witness, t ends in $R_t\psi$ whatever v held, so $R_t^\dagger$ returns it to ψ without touching v, and if $p \equiv \mathcal{S}(R, U)$ the same $R_t^\dagger$ restores p's temporary, because a context may itself apply $R_t^\dagger$ and look at t. For program (a) it is H, and Grover's diffusion [9, §4] is $-T$ when w is uniform.

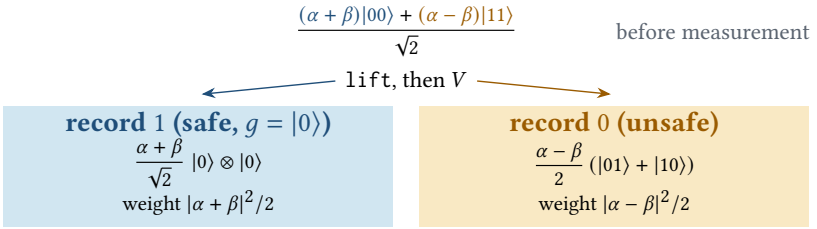
3 One measurement record at a time

With measurement, a program can be safe on one sequence of outcomes and unsafe on another. Our `lift` measures a register and, as dynamic lifting [8] does but with no separate circuit-generation stage, binds an outcome that only classical `if` guards can read. In `lift[{0}] t`, outcome 1 keeps the coordinates where t reads 0 and outcome 0 keeps their complement. A *record* o lists the outcomes along one path through the program, and its exposed map K_o is the joint map along that path, left unnormalized on purpose, so that at a normalized input x the probability of reaching the drop along that path is $\|K_o x\|^2$.

```
prog c (v : [2]) : [2] {
  new t : [2] @ |0>;
  H v; CX v t;
  b = lift[{0}] t;
  V t v; drop t; return v }
```

```
prog witness (v : [2]) : [2] {
  new t : [2] @ |0>; new a : [2] @ |0>;
  H v; CX v a;
  k = lift[{0}] a; drop a;
  drop t; return v }
```

Take the listed program (c) at input $x = \alpha|0\rangle + \beta|1\rangle$, with t first. Before the measurement it holds $((\alpha+\beta)|00\rangle + (\alpha-\beta)|11\rangle)/\sqrt{2}$, and V fixes $|00\rangle$ and sends $|11\rangle$ to $(|01\rangle + |10\rangle)/\sqrt{2}$, so the two records leave the unnormalized states $K_o x$, $o = 1, 0$, pictured with their weights.



On record 1 the slices are again proportional, with $W_1 x = (\alpha+\beta)|0\rangle/\sqrt{2}$, while record 0 holds a Bell state whenever $\alpha \neq \beta$ and is unsafe. Normalizing $W_1 x$ would hide record 1's weight, 1 at input $|+\rangle$ and only $1/2$ at input $|0\rangle$. To keep these weights, the measurement-free witness no longer suffices, because allocations and gates preserve length and make its map isometric, while W_1 shrinks some vector and a *contraction* lengthens none. The witness, with

$R_t = I$, puts the lost length on a fresh qubit a, preparing with U on (a, v) what p held on (t, v), and `lift[{0}] a` keeps outcome $k = 1$, where a reads 0, leaving $W_1 x$ beside $|0\rangle_t$ with the same weight. Any contraction W on v arises this way, as $x \mapsto |0\rangle \otimes Wx + |1\rangle \otimes (I - W^\dagger W)^{1/2} x$ preserves length and extends to a gate. By $p \equiv_o \mathcal{S}(R, U)$ we mean equal contextual observations on p's record o and on the witness's own kept outcome 1, omitting its fixed $|0\rangle_a$ factor, with neither run normalized, whatever happens on other records.

Theorem 3.1 (on a selected record). *Let the site s drop only allocated temporaries in a well-typed program p, and let o be a record whose path executes s. With every unitary available as a gate, let K_o be the exposed prefix's map at s on o, with the block first. The following are equivalent.*

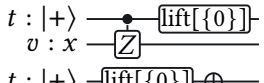
(1) $p \equiv_o \mathcal{S}(R, U)$ for some one-lift independent witness.

(2) There are a unit vector g on the block and a linear contraction W_o to all other registers such that $\forall x,\ K_o x = g \otimes W_o x$.

(3) One unitary on the block alone restores its pure allocation state ψ at every input on o.

If $K_o = 0$, all three conditions hold. Different records may need different return gates.

A caller who ignores the record sees only the *summed channel* [26, §2.2.1], which averages the records' states of the returned registers and a reference by their probabilities. It predicts every joint measurement of both, but it cannot decide safety on each record. For example, run the programs drawn below at input $x = \alpha|0\rangle + \beta|1\rangle$. Before `drop t` their unnormalized states on records 1 and 0 are



$|0\rangle \otimes x/\sqrt{2}$ $\quad$ $|1\rangle \otimes Zx/\sqrt{2}$ $\quad$ safe on both



$(\alpha|00\rangle + \beta|11\rangle)/\sqrt{2}$ $\quad$ $(\alpha|10\rangle + \beta|01\rangle)/\sqrt{2}$ $\quad$ safe on neither

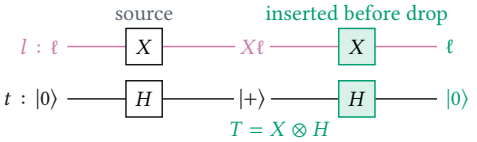
Only the first row gives t one input-independent state per record. Yet both act on v as the same summed channel, measuring v as $|0\rangle$ or $|1\rangle$ with probabilities $|\alpha|^2$ and $|\beta|^2$, and every record has probability $1/2$ at every input, so only per-record states tell them apart.

4 Blocks and loans

Dropping a tuple tests the named registers together. A Bell pair prepared in two fresh temporaries and left untouched is safe as a block of two, since the pair has one fixed state vector, though each half alone has the mixed state $I/2$ and so supplies no pure certificate.

We also consider blocks containing *loans*, registers borrowed in unknown states, possibly entangled with their caller, who expects them back as they were. The witness cannot prepare such a block, since it does not know the loan's state ℓ. Instead $\mathcal{S}(R, U)$ receives ℓ, allocates the temporaries beside it (the map $J\ell = \ell \otimes \psi$ with loans first), applies R to the whole block, and lets U act only outside it. The fixed certificate g, a map from $\mathbb{C}$ into the block, becomes an isometry $B : \mathscr{H}_{\text{loan}} \to \mathscr{H}$ from the loan's space into the block's space $\mathscr{H}$, carrying the loan's input along. The drop is then safe, $p \equiv \mathcal{S}(R, U)$, exactly when $K = B \otimes W$. The witness exists because a unitary R of the block carries the isometry J onto B, $RJ = B$, and the return gate $T = R^\dagger$ satisfies $TB = J$. For program `mixed`, listed beside its circuit, with loan l, we have $B\ell = X\ell \otimes |+\rangle$, and the inserted $T = X \otimes H$ hands the unknown loan back to the caller at the drop and restores t, although `mixed` returns `()`.

```
prog mixed (loan l : [2]) : 1 {
  new t : [2] @ |0>;
  H t; X l;
  drop (l,t); return () }
```



With measurement we again compare one record at a time, using $B \otimes W_o$, where B and the return gate may depend on the record. If the caller's loan and reference together are correlated with the program's other inputs, the record can change their normalized joint state, which the return gate does not undo. If they are not, or if the program measures nothing, the gate restores that state on every record that occurs.

5 Scope and further work

The language is linear and first order. We prove progress and preservation for its abstract machine with lifting and classical guards, and extend it to restore a safe block on each record and pool its temporaries for a later `new` of matching dimension and state. Reuse preserves the records, their weights and, on each record, the reduced state of the returned registers. Our Haskell checkers compute exactly: one builds K_o at one-temporary sites, the other runs the program on basis inputs and asks that a block of temporaries end in one state per record, up to phase, on every run reaching the drop. Both are exponential in the number of qubits, but even with only basis allocations and X, `CX` and Toffoli gates, deciding whether a drop of temporaries is safe on a given record is coNP-complete in the size of the program text, as is deciding safety of all such drops on all records (compare [14, Cor. 3.1]).

Our OpenQASM 3 compiler uses the second check, computes return gates, and reuses basis-allocated blocks safe on every record, clearing them to $|0\rangle$. Changing program (a)'s return and type to `new u : [2] @ |1>; return (v,u)` uses two circuit qubits, like a reset, where fresh allocation uses three: the return gate restores t before X prepares u on t's wire. Only basis allocations compile, program (c)'s V is unsupported, and loans drop only before lifts or classical guards. The loan-return check tests $TB = J$ on rounded columns of B.

Testing what a temporary ends as, not how it arose, our criterion accepts the Deutsch–Jozsa drop that Silq and Unqomp refuse and Qutes [7] does not reclaim, while reversing program (b)'s gates changes what it returns. Qurts puts lifetimes in types for uncomputation [11, §3] and Reqomp trades qubits for gates [18, §4], but both uncompute. ReQWIRE verifies that an ancilla is restored [20, §4], as Q# [16] and fractional types [6] require, and Qutes inverts a temporary's history once it is dead or output-isolable, which Deutsch–Jozsa's balanced oracle is not [7, Defs. 2–6]. Local rotations repair separable errors on a dirty ancilla [13, §5.2, Alg. 2], where our one gate per record restores a whole block. On measurement-free circuits with $|0\rangle$ ancillas, the validity test of slices summing to a unitary [14, Def. 3.1] is incomparable with safety: those of program (a) sum to $\sqrt{2}I$, proportional to a unitary but not norm preserving, yet its drop is safe, and those of `new t : [2] @ |0>; CX v t; drop t` sum to I though its drop is unsafe. QBorrow needs an identity loan factor per denoted operation [24, Def. 5.1], Bona one per borrowing body [27, Def. 3.2], where our return gate undoes any isometric encoding B. Separation logics prove loan returns by hand [15, 25]. Loops and quantum recursion, which re-enter scopes, remain open, as do their certificates and exact or approximate returns in a finite gate set.

References

[1] Samson Abramsky and Bob Coecke. 2004. A Categorical Semantics of Quantum Protocols. In *Proceedings of the 19th Annual IEEE Symposium on Logic in Computer Science (LICS 2004)*. 415–425. doi:10.1109/LICS.2004.1319636

[2] Charles H. Bennett. 1973. Logical Reversibility of Computation. *IBM Journal of Research and Development* 17, 6 (1973), 525–532. doi:10.1147/rd.176.0525

[3] Charles H. Bennett. 1989. Time/Space Trade-Offs for Reversible Computation. *SIAM J. Comput.* 18, 4 (1989), 766–776. doi:10.1137/0218053

[4] Benjamin Bichsel, Maximilian Baader, Timon Gehr, and Martin Vechev. 2020. Silq: A High-Level Quantum Language with Safe Uncomputation and Intuitive Semantics. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '20)*. ACM, 286–300. doi:10.1145/3385412.3386007

[5] Jacques Carette, Chris Heunen, Robin Kaarsgaard, and Amr Sabry. 2024. How to Bake a Quantum Π. *Proceedings of the ACM on Programming Languages* 8, ICFP (2024), 1–29. doi:10.1145/3674625

[6] Chao-Hong Chen, Vikraman Choudhury, Jacques Carette, and Amr Sabry. 2020. Fractional Types: Expressive and Safe Space Management for Ancilla Bits. In *Reversible Computation — 12th International Conference, RC 2020 (Lecture Notes in Computer Science, Vol. 12227)*. Springer, 169–186. doi:10.1007/978-3-030-52482-1_10

[7] Simone Faro, Francesco Pio Marino, and Gabriele Messina. 2026. Reversible Lifetime Semantics for Quantum Programs. arXiv:2603.14538 [quant-ph] (numbered as in version 1).

[8] Peng Fu, Kohei Kishida, Neil J. Ross, and Peter Selinger. 2023. Proto-Quipper with Dynamic Lifting. *Proceedings of the ACM on Programming Languages* 7, POPL, Article 11 (2023), 26 pages. doi:10.1145/3571204

[9] Lov K. Grover. 1996. A Fast Quantum Mechanical Algorithm for Database Search. In *Proceedings of the Twenty-Eighth Annual ACM Symposium on the Theory of Computing (STOC)*. ACM, 212–219. doi:10.1145/237814.237866

[10] Chris Heunen and Robin Kaarsgaard. 2022. Quantum Information Effects. *Proceedings of the ACM on Programming Languages* 6, POPL, Article 2 (2022). doi:10.1145/3498663

[11] Kengo Hirata and Chris Heunen. 2025. Qurts: Automatic Quantum Uncomputation by Affine Types with Lifetime. *Proceedings of the ACM on Programming Languages* 9, POPL, Article 6 (2025), 28 pages. doi:10.1145/3704842

[12] Alston S. Householder. 1958. Unitary Triangularization of a Nonsymmetric Matrix. *J. ACM* 5, 4 (Oct. 1958), 339–342. doi:10.1145/320941.320947

[13] Jiqi Li, Jingyi Mei, Wang Fang, and Ji Guan. 2026. Formal Verification of Quantum Ancilla Safety. In *Computer Aided Verification*. Springer Nature Switzerland, 326–348. doi:10.1007/978-3-032-32537-2_16 (numbered as in arXiv:2608.13099v1).

[14] Chenke Liu, Li Zhou, and Boning Meng. 2026. Quantum Uncomputation of Clean and Dirty Ancilla Qubits. *Proceedings of the ACM on Programming Languages* 10, OOPSLA2 (2026), 2270–2298. doi:10.1145/3839528 (numbered as in arXiv:2608.09578v2).

[15] Yusuke Matsushita, Kengo Hirata, Ryo Wakizaka, and Emanuele D'Osualdo. 2026. RapunSL: Untangling Quantum Computing with Separation, Linear Combination and Mixing. *Proceedings of the ACM on Programming Languages* 10, POPL, Article 6 (2026). doi:10.1145/3776648

[16] Microsoft. 2025. Quantum Memory Management in Q#. Microsoft Learn, Azure Quantum documentation. https://learn.microsoft.com/en-us/azure/quantum/user-guide/language/statements/quantummemorymanagement

[17] Anouk Paradis, Benjamin Bichsel, Samuel Steffen, and Martin Vechev. 2021. Unqomp: Synthesizing Uncomputation in Quantum Circuits. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI '21)*. ACM, 222–236. doi:10.1145/3453483.3454040

[18] Anouk Paradis, Benjamin Bichsel, and Martin Vechev. 2024. Reqomp: Space-Constrained Uncomputation for Quantum Circuits. *Quantum* 8 (2024), 1258. doi:10.22331/q-2024-02-19-1258

[19] Jennifer Paykin, Robert Rand, and Steve Zdancewic. 2017. QWIRE: A Core Language for Quantum Circuits. In *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages (POPL '17)*. 846–858. doi:10.1145/3009837.3009894

[20] Robert Rand, Jennifer Paykin, Dong-Ho Lee, and Steve Zdancewic. 2019. ReQWIRE: Reasoning about Reversible Quantum Circuits. *Electronic Proceedings in Theoretical Computer Science* 287 (2019), 299–312. doi:10.4204/EPTCS.287.17

[21] Peter Selinger. 2004. Towards a Quantum Programming Language. *Mathematical Structures in Computer Science* 14, 4 (2004), 527–586. doi:10.1017/S0960129504004256

[22] Peter Selinger and Benoît Valiron. 2006. A Lambda Calculus for Quantum Computation with Classical Control. *Mathematical Structures in Computer Science* 16, 3 (2006), 527–552. doi:10.1017/S0960129506005238

[23] Sam Staton. 2015. Algebraic Effects, Linearity, and Quantum Programming Languages. In *Proceedings of the 42nd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '15)*. 395–406. doi:10.1145/2676726.2676999

[24] Bonan Su, Li Zhou, Yuan Feng, and Mingsheng Ying. 2026. Borrowing Dirty Qubits in Quantum Programs. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '26)*. 274–289. doi:10.1145/3779212.3790134 (numbered as in arXiv:2508.17190v1).

[25] Bonan Su, Li Zhou, Yuan Feng, and Mingsheng Ying. 2026. An Interpretation of Bunched Logic for Reasoning about Heap-Manipulating Quantum Programs. arXiv:2409.10153 [cs.PL] v2, 5 Aug 2026; v1 (16 Sep 2024) was titled "BI-based Reasoning about Quantum Programs with Heap Manipulations".

[26] John Watrous. 2018. *The Theory of Quantum Information*. Cambridge University Press. doi:10.1017/9781316848142 (numbered as in the 2018 prepublication draft).

[27] Xiaoquan Xu, Chenke Liu, Boning Meng, Zihao Shen, and Li Zhou. 2026. Bona: Automatic Management of Dirty Ancilla Borrowing in Quantum Circuits. arXiv:2608.08765 [cs.PL] (numbered as in version 1).